Report:

- **Logs:** Every time you do something on Linux (like type a wrong password or plug in a USB), the computer writes a "sticky note" to a text file.
    - **The Problem:** These notes are messy, they are scattered in different folders, and there are thousands of them.
    - **The Goal:** Build a machine that picks up these notes, reads them, and tells you if something is wrong.

The "Assembly Line" (How it works)

There are 3 main "Stations" in your factory:

- **Station A (The Collector):** A script that goes to the secret Linux folders (like /var/log), grabs the notes, and puts them on the conveyor belt.
- **Station B (The Sorter):** This script looks at a messy note like: Apr 25 10:15:01 server1 sshd[123]: Failed password for root ...and picks out the important parts: **[Time: 10:15]**, **[User: root]**, **[Status: Failed]**.
- **Station C (The Warehouse):** This takes those organized parts and saves them into a **Database** (a digital table) so you can look at them later.

Positions:

Student 1: The Sorter (Data Science)

- **Your Task:** You focus on **Station B**.
- **What you do:** You use Python to take a line of text and "split" it into categories (Date, Type of Error, etc.). You are the "brain" that understands the text.

Student 2: The Warehouse Manager (Data Science)

- **Your Task:** You focus on **Station C**.
- **What you do:** You set up a simple table (SQLite). You write the code that takes the "sorted" data from Student 1 and saves it forever into that table.

Student 3: The Guard & Collector (Cyber Security)

- **Your Task:** You focus on **Station A** and the **Alarms**.
- **What you do:**
    1. Write the Linux command to copy the logs.
    2. Write a script that looks at the "Sorted" data and screams **"ALARM!"** if it sees the word "Failed" more than 3 times.

Week 1: The "Pipeline" Setup

The goal is to get the data moving and the "filing cabinet" (database) built.

Student	Sequential (Blocking) Tasks	Parallel (Independent) Tasks
Cyber Student	The Big Blocker: Create the collect_logs.sh script to copy logs from /var/log/auth.log. If you don't do this, the others have no data to work on.	Set up the Linux folder structure and permissions so the team can share files securely.
DS Student A	Create the **Database Schema** (The table structure). Define columns: Date, User, Status, Source_IP.	Practice using grep and awk on a sample text file to learn how to pick out words.

Student	Sequential (Blocking) Tasks	Parallel (Independent) Tasks
DS Student B	Create the **Empty SQLite Database** file using the schema from Student A.	Research common log formats so you know what "Normal" vs "Failed" looks like in text.

Week 2: The "Processing" Logic

The goal is to turn the messy text into organized information.

Student	Sequential (Blocking) Tasks	Parallel (Independent) Tasks
DS Student A	**The Logic Blocker:** Write the parser.py script. It takes the raw log line and splits it into the categories defined in Week 1.	Write a "Data Cleaning" rule: e.g., if a log line is empty or corrupted, the script should skip it.
DS Student B	**The Storage Blocker:** Write the db_insert.py script. This takes the "pieces" from Student A and saves them into the Database.	Write a simple "Search" query to prove that the database can show you only "Failed" attempts.
Cyber Student	**The Security Check:** Write a script that checks if the log file size has changed (to detect if the system is actually generating new logs).	Write the "Emergency Alert" logic: A script that prints **"WARNING"** to the screen if a specific IP appears too many times.

Week 3: The "Final Product" & Automation

The goal is to make it run by itself and show the results.

Student	Sequential (Blocking) Tasks	Parallel (Independent) Tasks
Cyber Student	**The Automation:** Set up the **Cron Job**. This tells Linux to run the Collector $\rightarrow$ Parser $\rightarrow$ DB Insert every 10 minutes.	Perform a "Security Test": Try to log in with a wrong password and see if your system catches you.
DS Student A	**The Reporting:** Create the generate_report.py script. It pulls stats from the database (e.g., "5 Errors today").	Format the report into a nice table so the Professor can read it easily.
DS Student B	**The Analytics:** Create a "Performance" script that finds the busiest hour of the day (when the most logs are created).	Create a simple chart (using Python) showing the number of logs per day for the week.

📂 Project File Structure

```
Safe-Vision-Log-System/
├── data/          # Folder for raw and processed logs
├── scripts/       # Where the code lives
```

```
│   ├── collect_logs.sh   # [Student: Cyber]
│   ├── parser.py        # [Student: DS-A]
│   ├── db_manager.py    # [Student: DS-B]
│   ├── analyzer.py      # [Student: DS-B]
│   └── main.sh          # [Student: Cyber]
├── reports/             # Folder for final summaries
└── logs_db.sqlite       # The database file
```